TECHNICAL DOUMENTATION

RAG-Based Customer Support Assistant using LangGraph

Prepared by:Thippani Srinija

Intern Id:IN226024402

Table of Contents

1 Introduction……..1

1.1 **What is RAG ……...1**
1.2 **Why it is needed……...1**
1.3 **Use case overview……1**
2 System Architecture 2
2.1 **Detailed explanation of HLD……..2**
2.2 **Componenets interactions……...2**
3 Design Decisions 3
3.1 **Chunck size choice……...3**
3.2 **Embedding strategy……3**
3.3 **Retrieval approach……...3**
3.4 **Prompt design logic……..3**
4 Workflow Explanation 4
4.1 **LangGraph usage……4**
4.2 **Node responsibilities……....4**
4.3 **State transitions …….……4**
5 Conditional Logic 5
5.1 **Intent detection……...5**
5.2 **Routing decisions……....5**
6 HITL Implementation 6
6.1 **Role of human intervention……...6**
6.2 **Benefits and Limitations……....6**
7 Challenges & Trade-offs 7
7.1 **Retreival accuracy vs speed……...7**
7.2 **Chunk size vs content quality……...7**
7.3 **Cost vs performance……...7-8**
8 Testing Strategy 9
8.1 **Testing approach……9**
8.2 **Sample queries……..9-10**
9 Future Enchancements 11
9.1 **Multi-document support……..11**
9.2 **Feedback loop……...11**
9.3 **Memory integration……11**
9.4 **Deployment……..11-12**

OBJECTIVE

The objective of this document is to provide a detailed technical explanation of the RAG-based customer support system. It describes how the system is designed, how different components interact, and how queries are processed to generate accurate responses.

1. INTRODUCTION

1.1 What is RAG?

Retrieval-Augmented Generation (RAG) is an advanced AI technique that combines information retrieval with response generation. Instead of generating answers purely from a trained model, RAG retrieves relevant information from an external knowledge source (such as a PDF) and uses it to produce accurate responses.

This approach ensures that answers are context-aware and fact-based.

1.2 Why it is Needed

Traditional chatbots rely only on pre-trained data, which leads to:

- Outdated information
- Incorrect or hallucinated responses
- Lack of domain-specific knowledge

RAG solves these issues by:

- Accessing real-time document data
- Reducing hallucination
- Improving accuracy and trust

1.3 Use Case Overview

This project implements a Customer Support Assistant that answers FAQs using a PDF document.

- User asks a question
- System retrieves relevant information
- Returns accurate answer
- Escalates to human if needed

Example:

User: What is return policy?
System: Customers can return products within 7 days if damaged or defective.

2. SYSTEM ARCHITECTURE EXPLANATION

2.1 Detailed Explanation of HLD

The system follows a layered architecture:

1. Data Layer → PDF document
2. Processing Layer → Chunking + Embeddings
3. Storage Layer → ChromaDB
4. Retrieval Layer → Fetch relevant chunks
5. Control Layer → LangGraph workflow
6. Response Layer → Output / HITL

Each layer performs a specific function to ensure modularity.

2.2 Component Interactions

PDF → Loader → Chunking → Embedding → Vector DB
User Query → Retriever → Relevant Chunks
↓
LangGraph
↓
Output / Human Support

All components work sequentially to produce the final response.

3. DESIGN DECISIONS

3.1 Chunk Size Choice

- Chunk size: 500
- Overlap: 100

Reason:

- Maintains context
- Improves retrieval accuracy
- Prevents loss of important information

3.2 Embedding Strategy

- Model: MiniLM
- Converts text into semantic vectors

Reason:

- Fast processing
- Lightweight
- Good performance for small datasets

3.3 Retrieval Approach

- Uses similarity search
- Top-k = 3 chunks

Reason:

- Ensures relevant data selection
- Reduces noise in results

3.4 Prompt Design Logic

Initially used LLM prompts, but:

- Generated long and incorrect answers
- Caused hallucination

Final approach:

- Direct retrieval-based answers
- Cleaner and more reliable output

4. WORKFLOW EXPLANATION

4.1 LangGraph Usage

LangGraph is used to manage system execution flow.

- Defines nodes and transitions
- Enables conditional routing
- Controls how data moves between components

4.2 Node Responsibilities

- **Process Node**
  - Retrieves relevant data
  - Generates answer
- **Output Node**
  - Displays answer to user
- **Human Node**
  - Handles escalation cases

4.3 State Transitions

State object:

{query, answer, confidence}

Flow:

Process → Output
Process → Human → Output

Ensures smooth data movement across nodes.

5. CONDITIONAL LOGIC

5.1 Intent Detection

The system analyzes the query to determine:

- What information is requested
- Which chunk is relevant

5.2 Routing Decisions

Based on confidence:

- High confidence → return answer
- Low confidence → escalate

5.3 Decision Factors

- Keyword matching
- Similarity score
- Availability of relevant data

Ensures correct routing of queries.

6. HITL IMPLEMENTATION

6.1 Role of Human Intervention

Human-in-the-loop ensures that the system remains reliable even when:

- Query is unclear
- No relevant data is found

● Process

System detects low confidence
↓
Requests human input
↓
Human provides answer
↓
Final answer returned

6.3 Benefits

- Improves accuracy
- Handles edge cases
- Builds trust

● Limitations

- Slower response time
- Requires human availability

7. CHALLENGES & TRADE-OFFS

7.1 Retrieval Accuracy vs Speed

In RAG systems, there is always a balance between accuracy and performance.

- Increasing the number of retrieved chunks (higher *k*) improves accuracy
- But it also increases processing time

In this project:

- A balanced value of **k = 3** is used
- Ensures fast response with sufficient accuracy

7.2 Chunk Size vs Context Quality

Chunk size plays a critical role in retrieval performance.

- Large chunks → better context but less precise matching
- Small chunks → precise matching but may lose context

Trade-off solution:

- Chunk size = **500 characters**
- Overlap = **100 characters**

This ensures:

- Context continuity
- Better semantic understanding

7.3 Cost vs Performance

Using advanced LLMs (like OpenAI GPT) improves output quality but:

- Requires API cost
- Increases latency

Decision in this project:

- Removed LLM for final answer generation
- Used direct retrieval

Result:

- Zero cost
- Faster response
- More accurate for FAQ-based system

8.TESTING STRATEGY

8.1 Testing Approach

The system is tested using **functional and scenario-based testing**.

Steps followed:

1. Load PDF and verify data extraction
2. Validate chunk creation
3. Check embedding generation
4. Test retrieval accuracy
5. Validate final output
6. Test HITL escalation

Ensures end-to-end system correctness

8.2 Sample Queries

The following queries were tested:

- What is return policy?
- How can I track my order?
- What payment methods are accepted?
- Can I cancel my order?
- Do you offer free shipping?

● **Expected Results**

- Correct answer retrieved from PDF
- No irrelevant or repeated data
- Proper routing (output / HITL)
- Fast response time

● **Testing Observations**

- System performs well for structured FAQs

- Retrieval is accurate
- No hallucination after removing LLM
- Minor noise removed using filtering logic

9. FUTURE ENCHANCEMENTS

9.1 Multi-Document Support

- Allow multiple PDFs as knowledge base
- Combine multiple sources
- Improve coverage of answers

9.2 Feedback Loop

- Allow users to rate responses
- Store feedback for improvement
- Improve retrieval over time

9.3 Memory Integration

- Store previous queries
- Enable conversational context
- Improve user experience

9.4 Deployment

- Convert system into web application
- Use frameworks like:
    - Streamlit
    - Flask / FastAPI
- Deploy on cloud platforms:
    - AWS
    - Azure
    - GCP

● Advanced Models

- Use better embedding models

- Integrate powerful LLMs (if needed)
- Improve understanding of complex queries

“The proposed system demonstrates a practical and scalable implementation of a RAG-based solution, effectively balancing accuracy, performance, and cost. With further enhancements, it can be extended into a production-level customer support system.”